

第7章 图论

7.1 深度优先遍历

深度优先遍历 (DFS) 不仅可用于搜索, 拓扑排序、欧拉回路、点/边双连通分量、强连通分量、2-SAT等也都是它的用武之地。

例7-1 【拓扑排序】给任务排序 (Ordering Tasks, UVa 10305)

假设有 n 个变量, 还有 m 个二元组 (u, v) , 分别表示变量 u 小于 v 。那么, 所有变量从小到大排列起来应该是什么样子呢? 例如, 有4个变量 a, b, c, d , 若已知 $a < b, c < b, d < c$, 则这4个变量的排序可能是 $a < d < c < b$ 。尽管还有其他可能 (如 $d < a < c < b$), 但是你只需找出其中一个即可。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;
const int NN = 104;
vector<int> G[NN];
int VIS[NN];

void dfs(int u, deque<int>& order) {
    VIS[u] = -1;
    for (auto v : G[u]) {
        assert(VIS[v] != -1);
        if (!VIS[v]) dfs(v, order);
    }
    VIS[u] = 1;
    order.push_front(u);
}

int main() {
    for (int n, m, u, v; scanf("%d%d", &n, &m) == 2 && n; ) {
        _for(i, 0, n) G[i].clear();
        _for(i, 0, m) {
            scanf("%d%d", &u, &v);
            G[u - 1].push_back(v - 1); // u < v, u-->v;
        }
        fill_n(VIS, NN, 0);
        deque<int> O;
        _for(u, 0, n) if (VIS[u] != 1) dfs(u, O);
        for (size_t i = 0; i < O.size(); i++)
            printf("%d%s", O[i] + 1, i == O.size() - 1 ? "\n" : " ");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-15
注意：本题使用了 deque 作为在头部快速插入元素 O(1) 的序列容器
同类题目：Spreadsheet Calculator, ACM/ICPC WF 1992, UVa 215
        Guess, UVa 1423
*/

```

例7-2 【欧拉回路】单词 (Play on Words, POJ 1386)

输入 n ($n \leq 100\,000$) 个单词，是否可以把所有这些单词排成一个序列，使得每个单词的第一个字母和上一个单词的最后一个字母相

同（如acm、malform、mouse）。每个单词最多包含1000个小写字母。输入中可以有重复单词。

【代码实现】

```

// 刘汝佳
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

const int maxn = 1000 + 4, SZ = 256;
int pa[SZ];           // 并查集
int findset(int x) { return pa[x] != x ? pa[x] = findset(pa[x]) : x; }
int used[SZ], deg[SZ]; // 字母出现过, 度数
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T, n;
    string s;
    cin >> T;
    while (T--) {
        cin >> n, fill_n(used, SZ, 0), fill_n(deg, SZ, 0);
        for (int ch = 'a'; ch <= 'z'; ch++) pa[ch] = ch; // 初始化并查集
        int cc = 26; // 连通块个数, 刚好是 26 个字母
        for (int i = 0; i < n; i++) {
            cin >> s;
            char c1 = s[0], c2 = s[s.size() - 1];
            deg[c1]++, deg[c2]--, used[c1] = used[c2] = 1;
            int s1 = findset(c1), s2 = findset(c2);
            if (s1 != s2) pa[s1] = s2, cc--;
        }

        vector<int> d;
        for (int ch = 'a'; ch <= 'z'; ch++) {
            if (!used[ch]) cc--; // 没出现过的字母
            else if (deg[ch] != 0) d.push_back(deg[ch]);
        }
        if (cc == 1 && (d.empty() || (d.size() == 2 && (d[0] == 1 || d[0] == -1))))
            cout << "Ordering is possible." << endl;
        else
            cout << "The door cannot be opened." << endl;
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题 6-16
注意: 本题使用了并查集来维护连通性
同类题目: The Necklace, UVa 10054
*/

```

例7-3 【欧拉道路】检查员的难题 (Inspector's Dilemma, ACM/ICPC Dhaka 2007, UVa 12118)

某国家有 V ($V \leq 1000$) 个城市，每两个城市之间都有一条双向道路直接相连，长度为 T 。你的任务是找一条最短的道路（起点和终点任意），使得该道路经过 E 条指定的边。

例如，若 $V=5$ ， $E=3$ ， $T=1$ ，指定的3条边分别为1-2、1-3和4-5，则最优道路为3-1-2-4-5，长度为 $4 \times 1 = 4$ 。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>

using namespace std;
typedef long long LL;
const int MAXV = 1000 + 4;
vector<int> G[MAXV];
int V, E, T, VIS[MAXV];

void dfs(int u, int& cnt) {
    VIS[u] = 1, cnt += G[u].size() % 2;
    for (size_t vi = 0; vi < G[u].size(); ++vi) {
        int v = G[u][vi];
        if (!VIS[v]) dfs(v, cnt);
    }
}

int main() {
    for (int k = 1; scanf("%d%d%d", &V, &E, &T) == 3 && (V || E || T); k++) {
        for (int i = 0; i < V; ++i) G[i].clear();
        fill_n(VIS, MAXV, 0);
        for (int i = 0, a, b; i < E; ++i) {
            scanf("%d%d", &a, &b), --a, --b;
            G[a].push_back(b), G[b].push_back(a);
        }
        int nc = 0, ans = 0; // 连通分量个数, 需要增加的边数
        for (int u = 0; u < V; ++u)
            if (!G[u].empty() && !VIS[u]) {
                int p = 0; // 奇度数点的个数
                dfs(u, p);
                if (p > 2) ans += (p - 2) / 2; // 增加(p-2)/2 条边才能保证存在欧拉道路
                nc++;
            }
        printf("Case %d: %d\n", k, T * (E + ans + max(0, nc - 1)));
    }
}
/*
算法分析请参考:《算法竞赛入门经典——习题与解答》习题 6-14
同类题目: Colored Sticks, POJ 2513
*/

```

例7-4 【点-双连通分量；二分图判定】圆桌骑士 (Knights of the Round Table, SPOJ KNIGHTS)

有 n ($1 \leq n \leq 1000$) 个骑士，他们经常举行圆桌会议，商讨大事。每次圆桌会议至少应有3个骑士参加，有 m ($1 \leq m \leq 10^6$) 对骑

士相互憎恨，且相互憎恨的骑士不能坐在圆桌旁的相邻位置。如果发生意见分歧，则需要举手表决，因此参加会议的骑士数目必须是奇数，以防止赞同票和反对票一样多。知道哪些骑士相互憎恨之后，你的任务是统计有多少个骑士不可能参加任何一个会议。

【代码实现】

```
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
struct Edge { int u, v; };
const int NN = 1000 + 10;
int pre[NN], iscut[NN], bccno[NN], dfs_clock, bcc_cnt; // 割顶的bccno无意义
vector<int> G[NN], bcc[NN];

stack<Edge> S;
int dfs(int u, int fa) {
    int lowu = pre[u] = ++dfs_clock, child = 0;
    for (int i = 0; i < G[u].size(); i++) {
        int v = G[u][i];
        Edge e = (Edge) {u, v};
        if (!pre[v]) { // 没有访问过v
            S.push(e), child++;
            int lowv = dfs(v, u);
            lowu = min(lowu, lowv); // 用后代的low函数更新自己
            if (lowv >= pre[u]) {
                iscut[u] = true, bcc[++bcc_cnt].clear();
                while (true) {
                    Edge x = S.top(); S.pop(); // 栈上是eBCC中的边，同一条边可能属于多个eBCC
                }
            }
        }
    }
    bcc[bcc_cnt].push_back(u);
    return lowu;
}
```

```

        if (bccno[x.u] != bcc_cnt)
            bcc[bcc_cnt].push_back(x.u), bccno[x.u] = bcc_cnt;
        if (bccno[x.v] != bcc_cnt)
            bcc[bcc_cnt].push_back(x.v), bccno[x.v] = bcc_cnt;
        if (x.u == u && x.v == v) break;
    }
}
}
else if (pre[v] < pre[u] && v != fa)
    S.push(e), lowu = min(lowu, pre[v]); // 用反向边更新自己
}
if (fa < 0 && child == 1) iscut[u] = false;
return lowu;
}

void find_bcc(int n) { // 调用结束后 s 保证为空, 所以不用清空
    fill_n(pre, n + 1, 0), fill_n(iscut, n + 1, 0), fill_n(bccno, n + 1, 0);
    dfs_clock = bcc_cnt = 0;
    for (int i = 0; i < n; i++)
        if (!pre[i]) dfs(i, -1);
}

int odd[NN], color[NN];
bool bipartite(int u, int b) {
    for (int i = 0; i < G[u].size(); i++) {
        int v = G[u][i];
        if (bccno[v] != b) continue;
        if (color[v] == color[u]) return false;
        if (!color[v]) {
            color[v] = 3 - color[u];
            if (!bipartite(v, b)) return false;
        }
    }
    return true;
}

int A[NN][NN];
int main() {
    for (int kase = 0, u, v, n, m; scanf("%d%d", &n, &m) == 2 && n; ) {
        for (int i = 0; i < n; i++) G[i].clear();
        memset(A, 0, sizeof(A));
        for (int i = 0; i < m; i++) {
            scanf("%d%d", &u, &v), u--, v--;
            A[u][v] = A[v][u] = 1;
        }
        for (int u = 0; u < n; u++)
            for (int v = u + 1; v < n; v++)
                if (!A[u][v]) G[u].push_back(v), G[v].push_back(u);
    }
}

```



```

find_bcc(n);
fill_n(odd, n + 1, 0);
for (int i = 1; i <= bcc_cnt; i++) {
    fill_n(color, n + 1, 0);
    for (int j = 0; j < bcc[i].size(); j++)
        bccno[bcc[i][j]] = i; // 主要是处理割顶
    int u = bcc[i][0];
    color[u] = 1;
    if (!bipartite(u, i))
        for (int j = 0; j < bcc[i].size(); j++) odd[bcc[i][j]] = 1;
}
int ans = n;
for (int i = 0; i < n; i++) if (odd[i]) ans--;
printf("%d\n", ans);
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.2.2 节例题 5
注意：本题在记录同一个边双连通分量中的所有点时，遍历了栈中所有的边
其实也可以在栈中记录点，但是遍历边的方法适用场景更广泛
因为有的题目可能会涉及处理其中的边
*/

```

例7-5 【点-双连通分量；割点】井下矿工（Mining Your Own Business, WF2011, SPOJ BUSINESS）

有一座地下的稀有金属矿，由 n （ $n \leq 50\,000$ ）条隧道和一些连接点组成，其中每条隧道连接两个连接点。任意两个连接点之间最多只有一条隧道。为了降低矿工的危险，你的任务是在一些连接点处安装太平井和相应的逃生装置，使得不管哪个连接点倒塌，不在此连接点的所有矿工都能到达太平井逃生（假定除倒塌的连接点不能通行外，其他隧道和连接点完好无损）。为了节约成本，你应当在尽量少的连接点处安装太平井。还需要计算出当太平井的数目最小时的安装方案总数。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
const int NN = 1e5 + 8;
typedef long long LL;
int Low[NN], Pre[NN], DfsClock, IsCut[NN], BccNo;
vector<int> G[NN], Bcc[NN];

stack<int> S;
void init(int &n) {
    for (int i = 1; i <= n; ++i)
        G[i].clear(), Pre[i] = 0, IsCut[i] = 0;
    n = BccNo = DfsClock = 0;
}
```

```

}

void tarjan(int u, int root) {
    Pre[u] = Low[u] = ++DfsClock, S.push(u);
    int child = 0;
    for (auto v : G[u]) {
        if (!Pre[v]) {
            tarjan(v, root);
            Low[u] = min(Low[u], Low[v]), child++;
            if (Low[v] == Pre[u]) { // 发现一个 BCC
                Bcc[++BccNo].clear();
                for (int x = 0; x != v; S.pop()) Bcc[BccNo].push_back(x = S.top());
                Bcc[BccNo].push_back(u);
            }
            if (u != root && Low[v] >= Pre[u]) IsCut[u] = 1;
        }
        else
            Low[u] = min(Low[u], Pre[v]);
    }
    if (u == root && child > 1) IsCut[u] = 1;
}

int main() {
    for (int kase = 1, n, m; scanf("%d", &m) == 1 && m; ++kase) {
        init(n);
        for (int i = 1, x, y; i <= m; ++i) {
            scanf("%d%d", &x, &y), n = max(n, max(x, y));
            G[x].push_back(y), G[y].push_back(x);
        }
        for (int i = 1; i <= n; ++i) if (!Pre[i]) tarjan(i, i);
        printf("Case %d:", kase);
        LL ans1 = 0, ans2 = 1;
        for (int i = 1; i <= BccNo; ++i) { // 点双连通分量-缩点
            int cutCnt = 0, sz = Bcc[i].size();
            for (int vi = 0; vi < sz; vi++) if (IsCut[Bcc[i][vi]]) cutCnt++;
            if (cutCnt == 0) ans1 += 2, ans2 *= 1LL * sz * (sz - 1) / 2;
            else if (cutCnt == 1) ans1++, ans2 *= sz - 1;
        }
        printf(" %lld %lld\n", ans1, ans2);
    }
    return 0;
}

/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 5.2.3 节例题 6
同类题目: Network, CEOI 1996, POJ 1144
           Electricity, POJ 2117
           TWO NODES, HDU 4587
*/

```

例 7-6 【SCC; SCC图】等价性证明 (Proving Equivalences, NWERC 2008, HDU 2767)

在数学中，我们常常需要完成若干个命题的等价性证明。比如，有4个命题 a, b, c, d ，我们证明 $a \hat{=} b$ ，然后 $b \hat{=} c$ ，最后 $c \hat{=} d$ 。注意，每次证明都是双向的，因此一共完成了6次推导。另一种方法是证明 $a \models b$ ，然后 $b \models c$ ，接着 $c \models d$ ，最后 $d \models a$ ，只需4次。现在你的任务是证明 n 个命题全部等价，且你的朋友已经为你做出了 m 次推导（每次推导的内容已知），你至少还需要做几次推导才能完成整个证明？

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
const int NN = 20000 + 10;
vector<int> G[NN];
int pre[NN], lowlink[NN], sccno[NN], dfs_clock, scc_cnt;
stack<int> S;
void dfs(int u) {
    int &lu = lowlink[u]; // 使用引用变量简化重复代码
    pre[u] = lu = ++dfs_clock, S.push(u);
    for (size_t i = 0; i < G[u].size(); i++) {
        int v = G[u][i];
        if (!pre[v]) dfs(v), lu = min(lu, lowlink[v]);
        else if (!sccno[v]) lu = min(lu, pre[v]);
    }
    if (lu == pre[u]) {
        scc_cnt++;
        for (int x = -1; x != u; S.pop()) x = S.top(), sccno[x] = scc_cnt;
    }
}

void find_scc(int n) {
    dfs_clock = scc_cnt = 0;
    fill_n(sccno, n, 0), fill_n(pre, n, 0);
    for (int i = 0; i < n; i++) if (!pre[i]) dfs(i);
}

bool in0[NN], out0[NN];
int solve(int n) {
    if (scc_cnt == 1) return 0;
    fill_n(in0 + 1, scc_cnt, true), fill_n(out0 + 1, scc_cnt, true);
    for (int u = 0; u < n; u++)
        for (size_t i = 0; i < G[u].size(); i++) {
            int v = G[u][i]; // u 所在 scc 到 v 所在 scc 的一条边, 更新对应的度数
            if (sccno[u] != sccno[v]) in0[sccno[v]] = out0[sccno[u]] = false;
        }
    return max(count(in0 + 1, in0 + scc_cnt + 1, true),
               count(out0 + 1, out0 + scc_cnt + 1, true));
}

```

```

int main() {
    int T, n, m;
    scanf("%d", &T);
    while (T--) {
        scanf("%d%d", &n, &m);
        for (int i = 0; i < n; i++) G[i].clear();
        for (int i = 0, u, v; i < m; i++)
            scanf("%d%d", &u, &v), G[u - 1].push_back(v - 1);
        find_scc(n);
        printf("%d\n", solve(n));
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.2.3 节例题 7
注意：统计度数为 0 的 SCC 个数时，使用 STL 中的 count 函数简化了代码
同类题目：Popular Cows, USACO 2003 Fall, POJ 2186
*/

```

例 7-7 【SCC; DAG 动态规划】最大团 (The Largest Clique, UVa 11324)

给出一张有向图 G ，求一个结点数最大的结点集，使得该结点集中任意两个结点 u 和 v 满足：要么 u 可以到达 v ，要么 v 可以达到 u (u 和 v 相互可达也可以)。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
const int NN = 1000 + 10;
vector<int> G[NN];
int pre[NN], lowlink[NN], sccno[NN], dfs_clock, scc_cnt;
stack<int> S;
void dfs(int u) {
    int& lu = lowlink[u];
    pre[u] = lu = ++dfs_clock, S.push(u);
    for (size_t i = 0; i < G[u].size(); i++) {
        int v = G[u][i];
        if (!pre[v])
            dfs(v), lu = min(lu, lowlink[v]);
        else if (!sccno[v])
            lu = min(lu, pre[v]);
    }
    if (lu == pre[u]) {
        scc_cnt++;
        for (int x = -1; x != u; S.pop()) sccno[x = S.top()] = scc_cnt;
    }
}

```

```

void find_scc(int n) {
    dfs_clock = scc_cnt = 0;
    fill_n(sccno, n, 0), fill_n(pre, n, 0);
    for (int i = 0; i < n; i++) if (!pre[i]) dfs(i);
}

int SccSz[NN], TG[NN][NN], D[NN];
int dp(int u) {
    int& d = D[u];
    if (d >= 0) return d;
    d = SccSz[u];
    for (int v = 1; v <= scc_cnt; v++)
        if (u != v && TG[u][v]) d = max(d, dp(v) + SccSz[u]);
    return d;
}

int main() {
    int T, n, m;
    scanf("%d", &T);
    while (T--) {
        scanf("%d%d", &n, &m);
        for (int i = 0; i < n; i++) G[i].clear();
        for (int i = 0; u, v; i < m; i++)
            scanf("%d%d", &u, &v), G[u - 1].push_back(v - 1);
        find_scc(n); // 查找强连通分量
        memset(TG, 0, sizeof(TG)), fill_n(SccSz, n + 1, 0);
        for (int i = 0; i < n; i++) {
            SccSz[sccno[i]]++; // 累加强连通分量大小（结点数）
            for (size_t j = 0; j < G[i].size(); j++)
                TG[sccno[i]][sccno[G[i][j]]] = 1; // 构造 SCC 图
        }
        int ans = 0;
        fill_n(D, n + 1, -1); // 初始化记忆化数组
        for (int i = 1; i <= scc_cnt; i++) // 注意，SCC 编号为 1~scc_cnt
            ans = max(ans, dp(i));
        printf("%d\n", ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.2.3 节例题 8
同类题目：最大半连通子图，ZJOI 2007，牛客 NC 20603
*/

```

例7-8 【2-SAT】宇航员分组 (Astronauts, UVa 1391)

有 A, B, C 3个任务要分配给 n ($1 \leq n \leq 100\ 000$) 个宇航员，其中每个宇航员恰好要分配一个任务。设所有 n 个宇航员的平均年龄为 x (年龄为 $0 \sim 200$ 的整数)，只有年龄大于或等于 x 的宇航员才能分配任务 A ，只有年龄严格小于 x 的宇航员才能分配任务 B ，而任务 C 没有限制。有 m ($1 \leq m \leq 100\ 000$) 对宇航员相互讨厌，因此不能分配到同一任务。编程找出一个满足上述所有要求的任务分配方案。

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cstring>
#include <vector>
using namespace std;

const int maxn = 1e5 + 8;

struct TwoSAT {
    int n;
    vector<int> G[maxn * 2];
    bool mark[maxn * 2];
    int S[maxn * 2], c;

    bool dfs(int x) {
        if (mark[x ^ 1]) return false;
        if (mark[x]) return true;
        mark[x] = true, S[c++] = x;
        for (size_t i = 0; i < G[x].size(); i++)
            if (!dfs(G[x][i])) return false;
        return true;
    }

    void init(int n) {
        this->n = n;
        for (int i = 0; i < n * 2; i++) G[i].clear();
        fill_n(mark, 2 * n + 1, 0);
    }

    void add_clause(int x, int xval, int y, int yval) { // x = xval 或 y = yval
        x = x * 2 + xval, y = y * 2 + yval;
        G[x ^ 1].push_back(y), G[y ^ 1].push_back(x);
    }

    bool solve() {
        for (int i = 0; i < n * 2; i += 2)
            if (!mark[i] && !mark[i + 1]) {
                c = 0;
                if (!dfs(i)) {
                    while (c > 0) mark[S[--c]] = false;
                    if (!dfs(i + 1)) return false;
                }
            }
    }
};

```

```

    }
    return true;
}
};

#include <algorithm> // 题目相关
int n, m, total_age, age[maxn];
int is_young(int x) { return age[x] * n < total_age; }
TwoSAT solver;
int main() {
    while (scanf("%d%d", &n, &m) == 2 && n) {
        total_age = 0;
        for (int i = 0; i < n; i++) scanf("%d", &age[i]), total_age += age[i];
        solver.init(n);
        for (int i = 0, a, b; i < m; i++) {
            scanf("%d%d", &a, &b), a--, b--;
            if (a == b) continue;
            solver.add_clause(a, 1, b, 1); // 不能同去任务 C
            if (is_young(a) == is_young(b)) // 同类宇航员
                solver.add_clause(a, 0, b, 0); // 不能同去任务 A 或者任务 B
        }
        if (!solver.solve()) {
            puts("No solution.");
            continue;
        }
        for (int i = 0; i < n; i++) { // 看看 x[i] 的值
            if (solver.mark[i * 2]) puts("C"); // false: 去任务 C
            else if (is_young(i)) puts("B"); // true: 年轻宇航员去任务 B
            else puts("A"); // true: 年长宇航员去任务 A
        }
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 5.2.4 节例题 10
同类题目: Now or later, UVa 1146
          游戏, NOI 2017, 牛客 NC 20802
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表7-1所示。

表7-1 深度优先遍历例题归纳